appendix B. OAuth2 grant types

After reading chapter 9, you might think that OAuth2 doesn't seem too complicated. After all, you have an authentication service that checks a user's credentials and issues a token back to the user. The token can, in turn, be presented every time the user wants to call a service protected by the OAuth2 server.

With the interconnected nature of the web and cloud-based applications, users have come to expect that they can securely share their data and integrate functionality between different applications owned by various services. This presents a unique challenge from a security perspective because you want to integrate across different applications while not forcing users to share their credentials with each application they want to integrate with.

OAuth2 is a flexible authorization framework that provides multiple mechanisms for applications to authenticate and authorize users without forcing them to share credentials. Unfortunately, it's also one of the reasons why OAuth2 is considered complicated. These authentication mechanisms are called *authentication grants*. OAuth2 has four forms of authentication grants that client applications can use to authenticate users, receive an access token, and then to validate that token. These grant types are

- Password
- Client credential
- Authorization code
- Implicit

In the following sections, we walk through the activities that occur during the execution of each of these OAuth2 grant flows. We also talk about when to use one grant type over another.

B.1 Password grant types

An OAuth2 password grant is probably the most straightforward grant type. You'll use this grant type when both the application and the services explicitly trust one another. For example, the O-stock web application and the licensing and organization services are

all owned by the same company (Optima Growth), so there's a natural trust relationship between them.

NOTE To be explicit, when we refer to a "natural trust relationship," we mean that the same organization completely owns the application and services. These are managed under the same policies and procedures.

When a natural trust relationship exists, there's little concern about exposing an OAuth2 access token to the calling application. For example, the O-stock web application can use the granted OAuth2 password to capture the user's credentials and directly authenticate against the OAuth2 service. Figure B.1 shows the password grant type in action between O-stock and the downstream services.

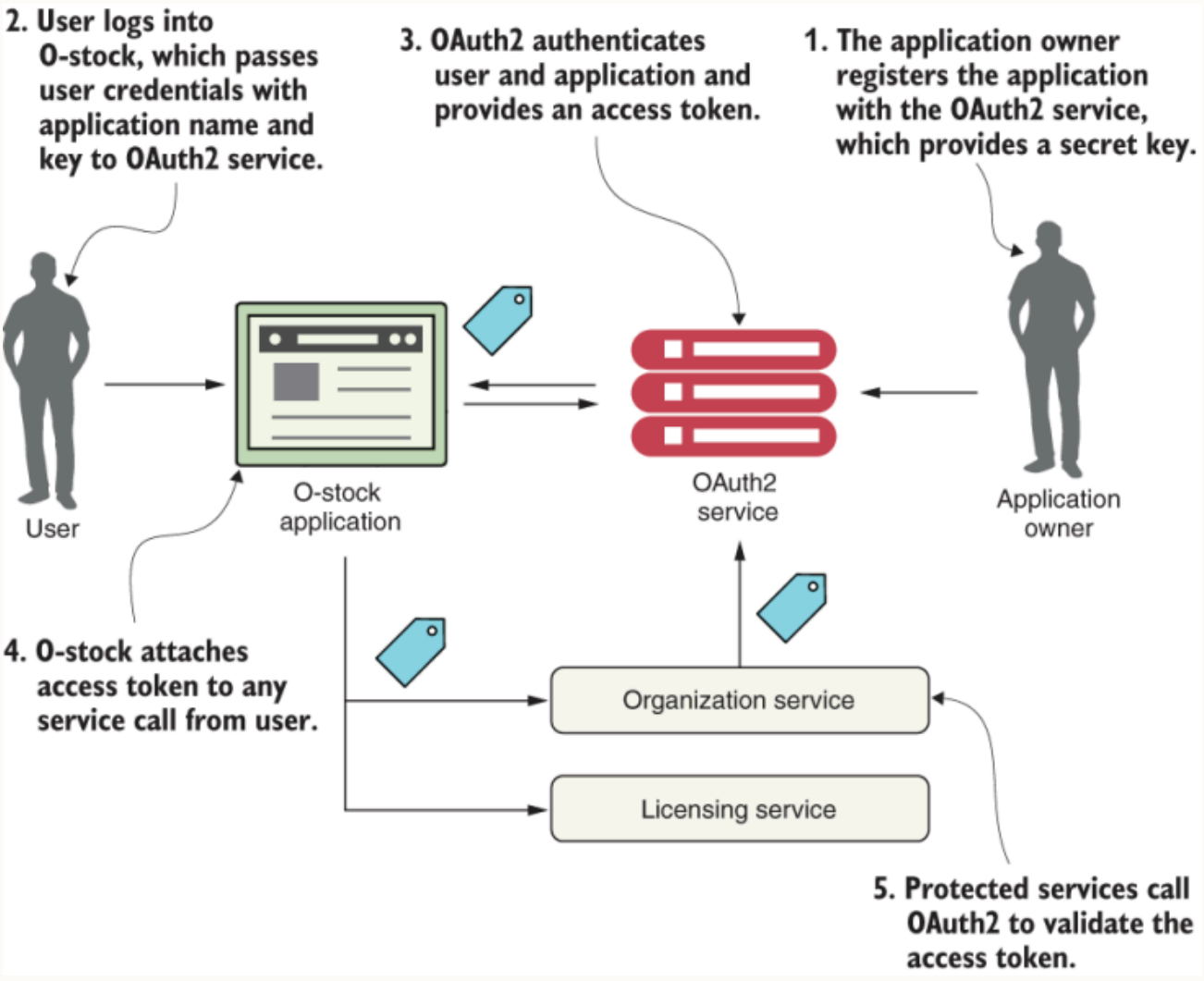


Figure B.1 The OAuth2 service determines if the user accessing the service is an authenticated user.

In figure B.1, the following actions take place. The numbers correspond to those in the figure:

1. Before the O-stock application can use a protected resource, it needs to be uniquely identified within the OAuth2 service. Usually, the owner of the application registers with the OAuth2 application service and provides a unique name for their application. The OAuth2 service then provides a secret key back to register the application. The name of the application and the secret key provided by the OAuth2 service uniquely identify the application trying to access any protected resources.
2. The user logs into O-stock and provides their login credentials to the application. O-stock passes the user credentials and the application name/application secret key directly to the OAuth2 service.
3. The O-stock OAuth2 service authenticates the application and the user and then provides an OAuth2 access token back to the user.
4. Every time the O-stock application calls a service on behalf of the user, it passes along the access token provided by the OAuth2 server.
5. When a protected service is called (in this case, one of the licensing or organization services), the service calls back to the O-stock OAuth2 service to validate the token. If the token is good, the invoked service allows the user to proceed. If the token is invalid, the OAuth2 service returns an HTTP status code 403, indicating that the token is invalid.

B.2 Client credential grant type

You'll use the client credential grant type when an application needs to access an OAuth2 protected resource, but no one is involved in the transaction. With the client credential grant type, the OAuth2 server only authenticates based on the application name and the secret key provided by the resource owner. This grant type is usually used when the same company owns both applications. The difference between the password grant type and the client credential grant type is that a client credential grant authenticates by only using the registered application name and the secret key.

For example, let's say that the O-stock application has a data analytics job that runs once an hour. As part of its work, it makes calls out to O-stock services. However, the O-stock developers still want that application to authenticate and authorize itself before it can access the data in those services. This is where the client credential grant type can be used. Figure B.2 shows this flow.

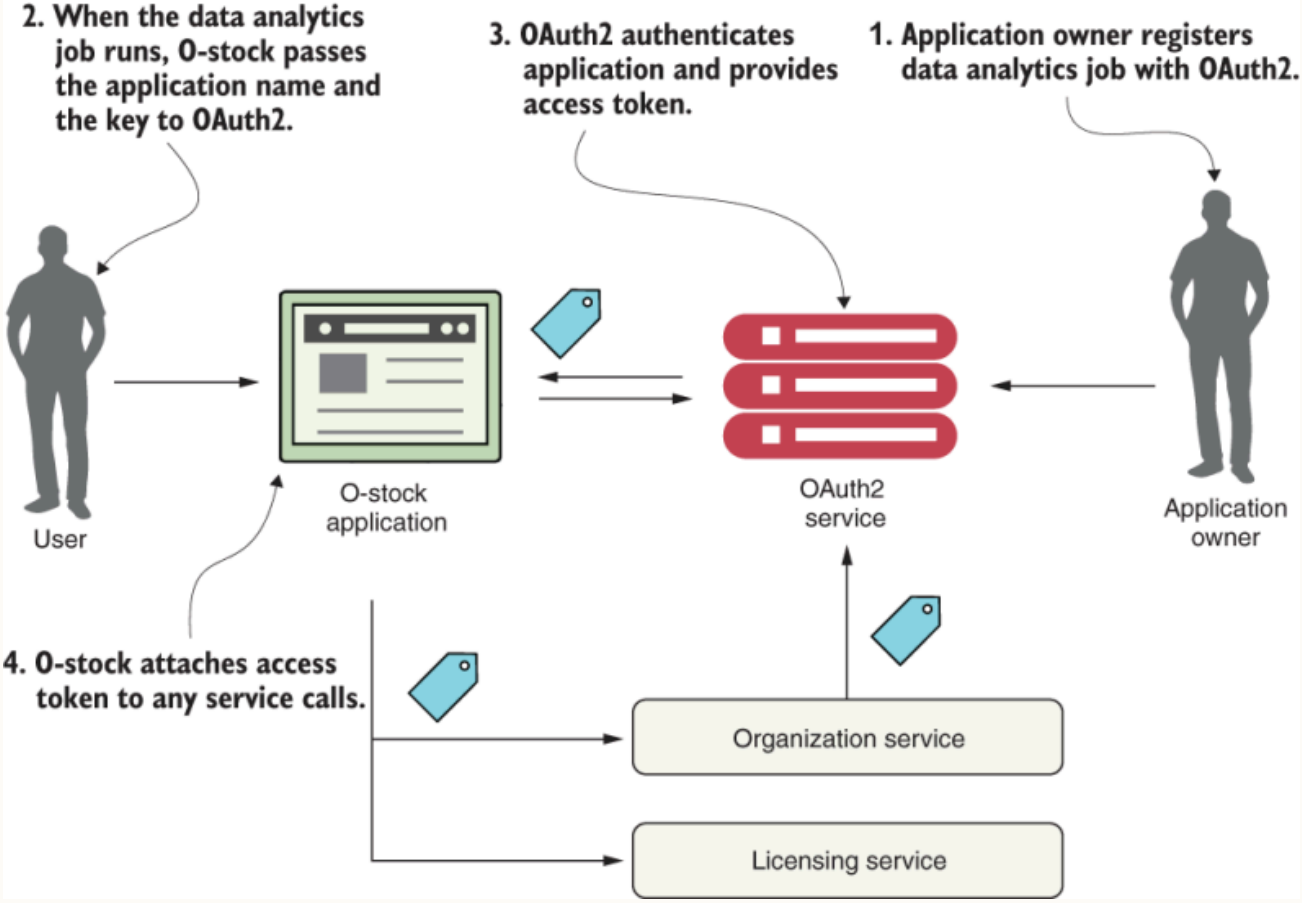


Figure B.2 The client credential grant is for "no-user-involved" application authentication and authorization.

In figure B.2, the following actions take place. The numbers correspond to those in the figure:

1. The resource owner registers the O-stock data analytics application with the OAuth2 service. The resource owner provides the application name and receives back a secret key.
2. When the O-stock data analytics job runs, it presents its application name and secret key provided by the resource owner.
3. The O-stock OAuth2 service authenticates the application using the provided application name and the secret key and then returns an OAuth2 access token.
4. Every time the application calls one of the O-stock services, it presents the OAuth2 access token received with the service call.

B.3 Authorization grant type

The authorization code grant is by far the most complicated of the OAuth2 grant types, but it's also the most common because it allows different applications from different vendors to share data and services without having to expose a user's credentials across multiple applications. It also enforces an extra layer of checking by not letting a calling

application get an OAuth2 access token immediately, but it rather receives a "preflight" authorization code.

The easiest way to understand the authorization grant type is with an example. Let's say you have an O-stock user who also uses Salesforce.com. The O-stock customer's IT department has built a Salesforce application that needs data from an O-stock service (the organization service). Let's look at figure B.3 to see how the authorization code grant type works to allow Salesforce access to data from the organization service without the O-stock customer ever having to expose their O-stock credentials to Salesforce.

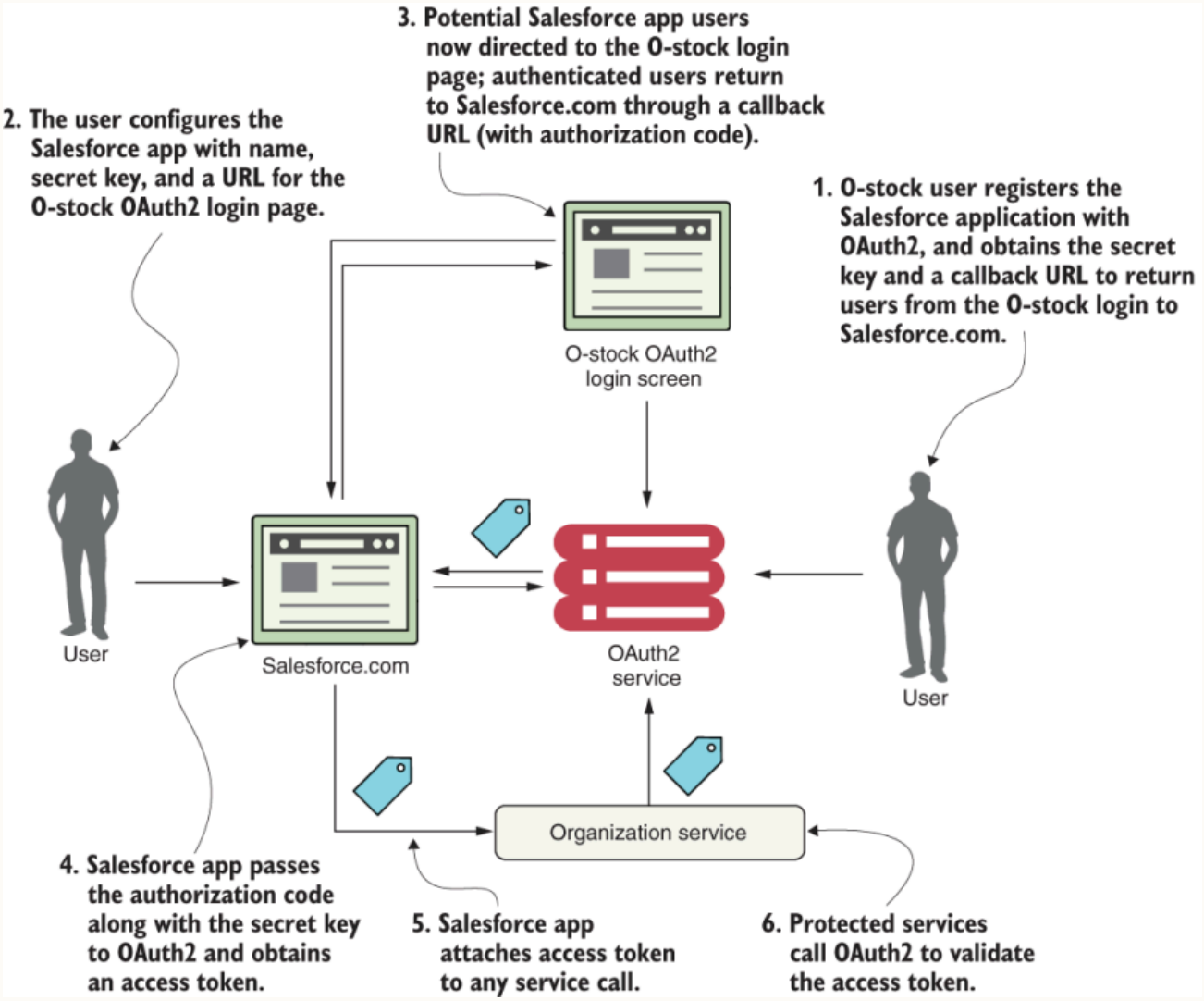


Figure B.3 The authentication code grant type allows applications to share data without exposing user credentials.

In figure B.3, the following actions take place. The numbers correspond to those in the figure:

1. The user logs in to the O-stock application and generates an application name and application secret key for the Salesforce application. As part of the registration process, they also provide a callback URL for the Salesforce application. This Salesforce callback URL is called after the OAuth2 server authenticates the O-stock credentials.
2. The user configures their Salesforce application with the following information:
    1. The application name they created for Salesforce
    2. The secret key they generated for Salesforce
    3. A URL that points to the O-stock OAuth2 login page

       Now when the user tries to use the Salesforce application and access O-stock data via the organization service, they are redirected to the O-stock login page via the URL described in step 1. The user provides their O-stock credentials. If they are valid credentials, the O-stock OAuth2 server generates an authorization code and redirects the user to Salesforce via the URL provided in step 1. The OAuth2 server sends the authorization code as a query parameter on the callback URL.
3. The custom Salesforce application persists the authorization code. Note that this authorization code isn't an OAuth2 access token.
4. Once the authorization code is stored, the Salesforce application presents the secret key generated during the registration process and the authorization code back to the O-stock OAuth2 server. The O-stock OAuth2 server validates the authorization code and then returns an OAuth2 token to the custom Salesforce application, which is used every time Salesforce needs to authenticate the user and get an OAuth2 access token.
5. The Salesforce application calls the O-stock organization service, passing an OAuth2 token in the header.
6. The organization service validates the OAuth2 access token passed into the O-stock service call. If the token is valid, the organization service processes the user's request.

Wow! We need to come up for air. Application-to-application integration is convoluted. The key takeaway from this entire process is that even though the user is logged in to Salesforce and accessing O-stock data, at no time were the user's O-stock credentials directly exposed to Salesforce. After the initial authorization code was generated by the OAuth2 service, the user never had to provide their credentials back to the O-stock service.

B.4 Implicit grant type

You use the implicit grant type when you're running a web application through a traditional server-side web programming environment like Java or .NET. What happens if

your client application is a pure JavaScript application or a mobile application that runs entirely in a web browser and doesn't rely on server-side calls to invoke third-party services? This is where the implicit grant type comes into play. Figure B.4 shows the general flow of what occurs with the implicit grant type.

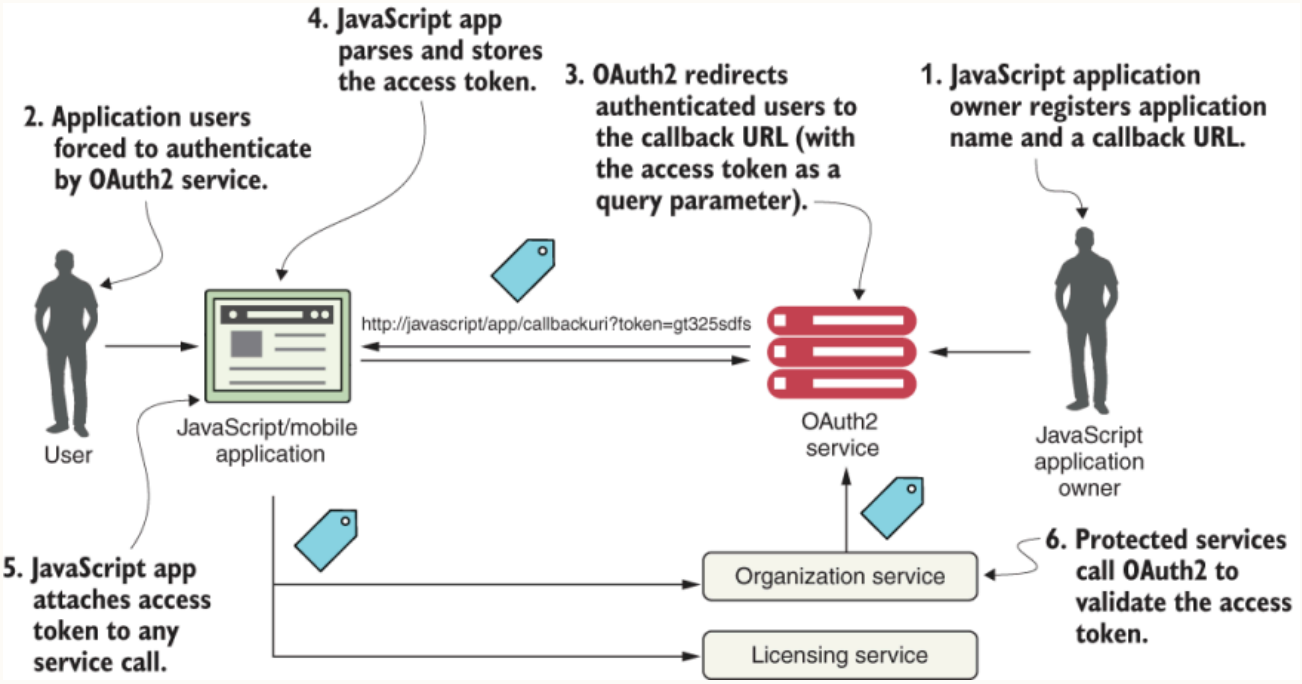


Figure B.4 The implicit grant type is used in a browser-based Single Page Application (SPA) JavaScript application.

With an implicit grant, you usually work with a pure JavaScript application running entirely inside a browser. With the other grant types, the client communicates with an application server that carries out the user's request, and the application server interacts with any downstream services. With an implicit grant type, all service interactions happen directly from the user's client (usually a web browser). In figure B.4, the following activities take place. Note that the numbers correspond to those in the figure:

1. The owner of the JavaScript application registers the application with the O-stock OAuth2 server. The owner provides an application name and a callback URL that will be redirected with the OAuth2 access token for the user.
2. The JavaScript application calls the OAuth2 service. The JavaScript application must present a preregistered application name. The OAuth2 server then forces the user to authenticate.
3. If the user successfully authenticates, the O-stock OAuth2 service doesn't return a token but, instead, redirects the user back to the page the owner of the JavaScript application registered in step 1. In the URL redirected back to the user, the OAuth2 access token is passed as a query parameter by the OAuth2 authentication service.
4. The application takes the incoming request and runs a JavaScript script that parses the OAuth2 access token and stores it (usually as a cookie).

5. Each time a protected resource is called, the OAuth2 access token is presented to the called service.
6. The called service validates the OAuth2 token and checks that the user is authorized to do the activity they're attempting to do.

Keep several things in mind regarding the OAuth2 implicit grant type. First, the implicit grant is the only grant type where the OAuth2 access token is directly exposed to a public client (web browser).

With the authorization grant type, the client application receives an authorization code returned by the application server hosting the application, and the user is granted an OAuth2 access by presenting the authorization code. The returned OAuth2 token is never directly exposed to the user's browser. With the client credential grant type, the grant occurs between two server-based applications, and with the password grant type, both the application requesting a service and the services are trusted and owned by the same organization.

OAuth2 tokens generated by the implicit grant type are more vulnerable to attack and misuse because the tokens are made available to the browser. Any malicious JavaScript running in the browser can access the OAuth2 access token and call the services you retrieved the OAuth2 token for on your behalf, essentially impersonating you. The implicit grant type OAuth2 tokens should therefore be short-lived (1–2 hours). Because the OAuth2 access token is stored in the browser, the OAuth2 spec (and Spring Cloud security) doesn't support the concept of a refresh token in which a token can be automatically renewed.

B.5 How tokens are refreshed

When an OAuth2 access token is issued, it is valid for a limited amount of time, eventually expiring. When the token expires, the calling application (and user) need to re-authenticate with the OAuth2 service. However, in most of the OAuth2 grant flows, the OAuth2 server will issue both an access token and a refresh token. A client can present the refresh token to the OAuth2 authentication service, and the service will validate the refresh token and then issue a new OAuth2 access token. Let's look at figure B.5 and walk through the token refresh flow.

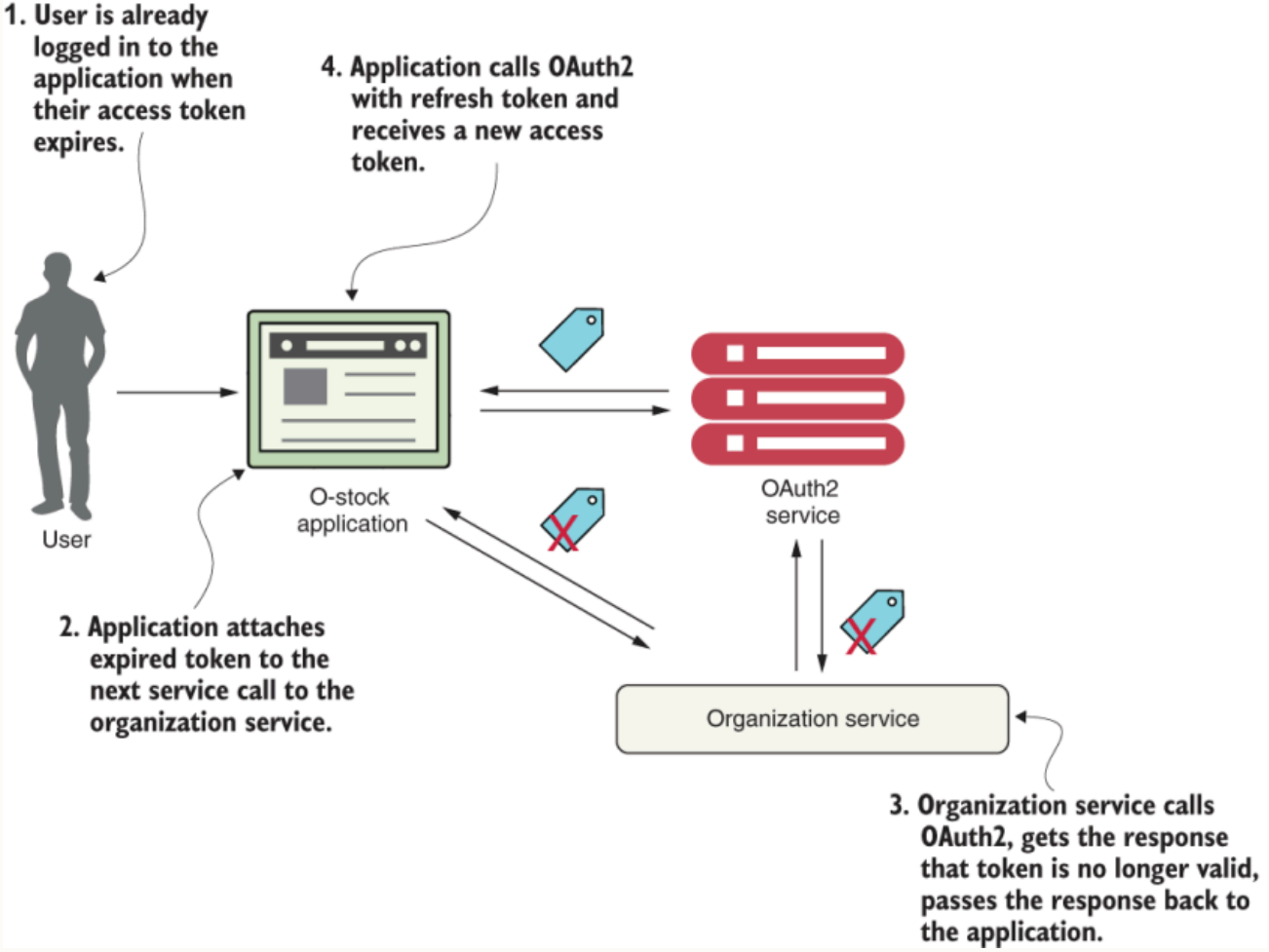


Figure B.5 The refresh flow allows an application to get a new access token without forcing the user to reauthenticate.

In figure B.5, the following actions take place. The numbers correspond to those in the figure:

1. The user logs in to O-stock but is already authenticated with the OAuth2 service. The user is happily working, but unfortunately, their token expires.
2. The next time the user tries to call a service (say, the organization service), the O-stock application passes the expired token to the organization service.
3. The organization service tries to validate the token with the OAuth2 service, which returns an HTTP status code 401 (unauthorized) and a JSON payload, indicating that the token is no longer valid. The organization service returns an HTTP 401 status code to the calling service.
4. The O-stock application gets the 401 HTTP status code and the JSON payload back from the organization service. The O-stock application then calls the OAuth2 authentication service with a refresh token. The OAuth2 authentication service validates the refresh token and then sends back a new access token.

Previous chapter

appendix A. Microservices architecture best practices

Next chapter

appendix C. Monitoring your microservices